



## Deep learning

### II Redes Neuronales: Perceptrón Multicapa para regresión MLP

#### Tarea 3

#### Predicción de precios de propiedades

El repositorio se obtuvo del siguiente enlace y se convirtió a CSV:

<http://lib.stat.cmu.edu/datasets/boston>

Con el repositorio Boston que está adjunto realice lo siguiente en un Notebook de Jupyter:

1. Lea la descripción de las columnas del dataset que está disponible en el enlace.  
La variable objetivo es MEDV.
2. Cargue los datos en Python usando Pandas y muestre los primeros 10 registros.
3. Muestre las dimensiones del dataframe y el tipo de las columnas (predictores).
4. Muestre los valores estadísticos básicos de todas las columnas (contador, media, desviación estándar, valor mínimo, valor máximo y los cuartiles).
5. Genere los gráficos por pares de todas las columnas con el siguiente comando:

```
sns.pairplot(datos)
```

Estos gráficos nos sirven para identificar la correlación que existe entre las variables.

6. Genere los gráficos de distribución de todas las variables con el siguiente código:

```
rows = 2
cols = 7

fig, ax = plt.subplots(nrows = rows , ncols = cols , figsize = (16,4))

col = datos.columns
index = 0

for i in range(rows):
    for j in range(cols):
        sns.histplot(datos[col[index]], ax = ax[i][j])
        index +=1
plt.tight_layout()
```

7. Muestre el mapa de calor de la matriz de correlación y su formato tidy ignorando la variable objetivo: **MEDV**.

*Revise el código del ejercicio de Propiedades de Brooklyn.*



8. Elimine las columnas que están altamente correlacionadas ( $>0.7$ ) ignorando la variable objetivo: **MEDV**.
9. Elimine las columnas de varianza baja (ninguna, pero realice la prueba).
10. Realice el gráfico de barras y el de bigotes de la variable objetivo.
11. Escale los datos con MinMaxScaler
12. Separe los datos en 80% entrenamiento y 20% para pruebas.
13. Haga 1 primer modelo de red neuronal con 1 sola capa y un número de neuronas entre 100 y 150, por ejemplo:

```
from sklearn.neural_network import MLPClassifier
nn = MLPClassifier(
    activation='relu',
    hidden_layer_sizes=(120),
    alpha=0.001,
    random_state=20,
    early_stopping=False)
nn.fit(X_train, y_train)
```

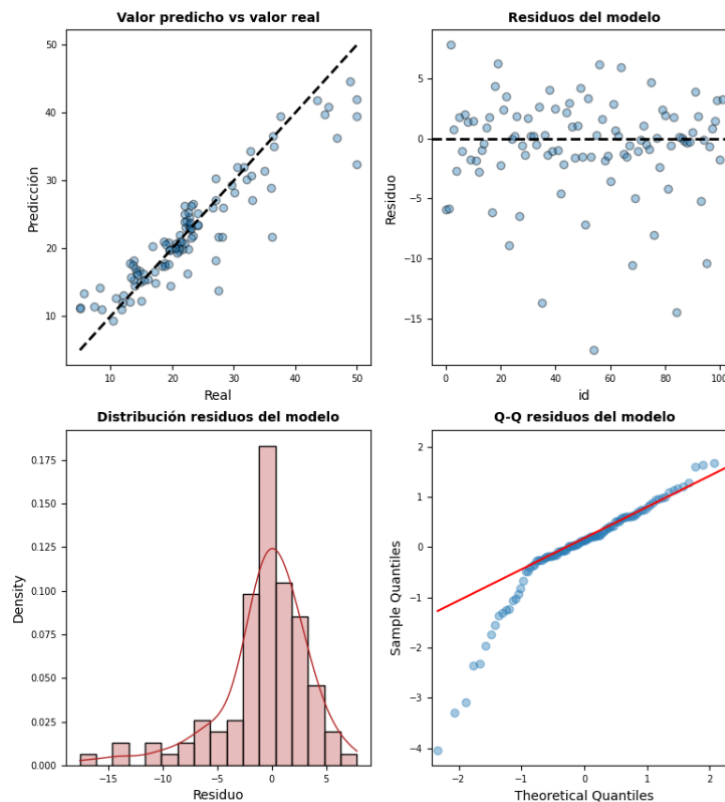
14. Realice las predicciones del modelo con los datos de **prueba (test)** y muestre el puntaje Rcuadrado y los errores RMSE, MAE y MSE.
15. Realice una búsqueda de la mejor configuración del modelo con GridSearchCV
  - usando al menos 3 valores para:  
'hidden\_layer\_sizes': [100, 120, 150, 180, 200, (30,30), (60,60), (80,80), (100,100)],
  - Usando un 'alpha': np.logspace(-3, 3, 6),
  - Usando al menos 3 valores para:  
'learning\_rate\_init': [0.00001, 0.0001, 0.001, 0.01],
  - Usando al menos 2 funciones de activación:  
'activation': ['tanh', 'relu', 'logistic']

Use el conjunto de **entrenamiento**.

16. Muestre los mejores 15 o 20 resultados ordenándolos por 'mean\_test\_score'.
17. Con el mejor modelo obtenido (modelo\_final = grid.best\_estimator\_) realice las predicciones del conjunto de **prueba**.
18. Obtenga el puntaje Rcuadrado y los errores RMSE, MAE y MSE.



19. Muestre las graficas que sirven para el diagnóstico de residuos.



20. Muestre los pesos y el bias del modelo.

21. Usando Keras genere una red neuronal con 2 capas ocultas que tengan entre 20 y 100 neuronas y la capa de salida que tenga 1 sola neurona.  
Puede elegir como función de activación: relu, tanh, softmax o sigmoid.  
Por ejemplo:

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense

model = Sequential([
    Dense(60, activation='tanh', input_shape=[9]),
    Dense(40, activation='relu'),
    Dense(1)
])
model.compile(loss='mean_squared_error', optimizer='adam',
              metrics=['mae', 'mse'])
```



```

model.fit(X_train, y_train, epochs=300)

# evaluamos el modelo
scores = model.evaluate(X_test, y_test)

```

- 22. Obtenga las predicciones del modelo con los datos de test y calcule el puntaje Rcuadrado y los errores MAE, MSE y RMSE.
- 23. Muestre las graficas que sirven para el diagnóstico de residuos.
- 24. Realice el diagrama del mejor modelo de red neuronal con viznet.
- 25. En un Excel que adjunte o una tabla html de jupyter-lab registre los mejores resultados de cada red neuronal.

| Modelo                             | Configuración                                                                                              | R^2 | RMSE | MAE |
|------------------------------------|------------------------------------------------------------------------------------------------------------|-----|------|-----|
| Mejor red neuronal<br>scikit-learn | Función activación: , learning rate:<br>Capa oculta 1: # neuronas                                          |     |      |     |
| Mejor red neuronal<br>Keras        | Capa oculta 1: # neuronas, función<br>Capa oculta 2: #neuronas, función<br>Capa salida: 1 neurona, función |     |      |     |

Como recordará este problema ya lo habíamos trabajado con árboles de regresión los cuales dieron un desempeño más bajo, esto es normal pues los árboles funcionan mejor en conjunto (Random Forest o Gradient Boost).

Resultados con árboles de regresión:

- r2\_score is 0.5564791971697227
- MAE is 3.892869274316428
- MSE is 35.01252486566836
- RMSE is 5.91713823276661